fitsyn.ipynb > plt.figure(figsize=(12,4))

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | Python

[5]

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

Python

[6]

```python
df=pd.read_csv("fitness_synthetic_dataset.csv")
```

Python

[4]

```python
df
```

Python

	timestamp	heart_rate	steps	sleep_hours
0	2026-01-01 00:00:00	47.2	0.0	6.60
1	2026-01-01 00:01:00	59.5	0.0	7.20
2	2026-01-01 00:02:00	60.5	0.0	6.55
3	2026-01-01 00:03:00	62.3	0.0	7.52
4	2026-01-01 00:04:00	51.7	0.0	5.77
...	...	...	...	...
1995	2026-01-02 09:15:00	83.6	26.0	0.00
1996	2026-01-02 09:16:00	74.8	19.0	0.00
1997	2026-01-02 09:17:00	67.9	22.0	0.00
1998	2026-01-02 09:18:00	73.7	26.0	0.00
1999	2026-01-02 09:19:00	NaN	18.0	0.00

2000 rows × 4 columns

[5]

```python
df.info()
```

Python

```
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 4 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   timestamp    2000 non-null   str
 1   heart_rate   1950 non-null   float64
 2   steps        1950 non-null   float64
 3   sleep_hours  2000 non-null   float64
dtypes: float64(3), str(1)
memory usage: 62.6 KB
```

[7]

```python
df['timestamp'] = pd.to_datetime(df['timestamp'])
df.set_index('timestamp', inplace=True)
```

Python

Generate + Code + Markdown

[7]

```python
df.info()
```

Python

```
<class 'pandas.DataFrame'>
DatetimeIndex: 2000 entries, 2026-01-01 00:00:00 to 2026-01-02 09:19:00
Data columns (total 3 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   heart_rate   1950 non-null   float64
 1   steps        1950 non-null   float64
 2   sleep_hours  2000 non-null   float64
dtypes: float64(3)
memory usage: 62.5 KB
```

```python
df.isnull().sum()
```

[8] Python

```
heart_rate     50
steps          50
sleep_hours     0
dtype: int64
```

```python
print((df.isnull().sum() / len(df)) * 100)
```

[9] Python

```
heart_rate     2.5
steps          2.5
sleep_hours    0.0
dtype: float64
```

```python
df.shape
```

[10] Python

```
(2000, 3)
```

```python
df['heart_rate'] = df['heart_rate'].interpolate()
df['steps'] = df['steps'].interpolate()
```

[8] Python

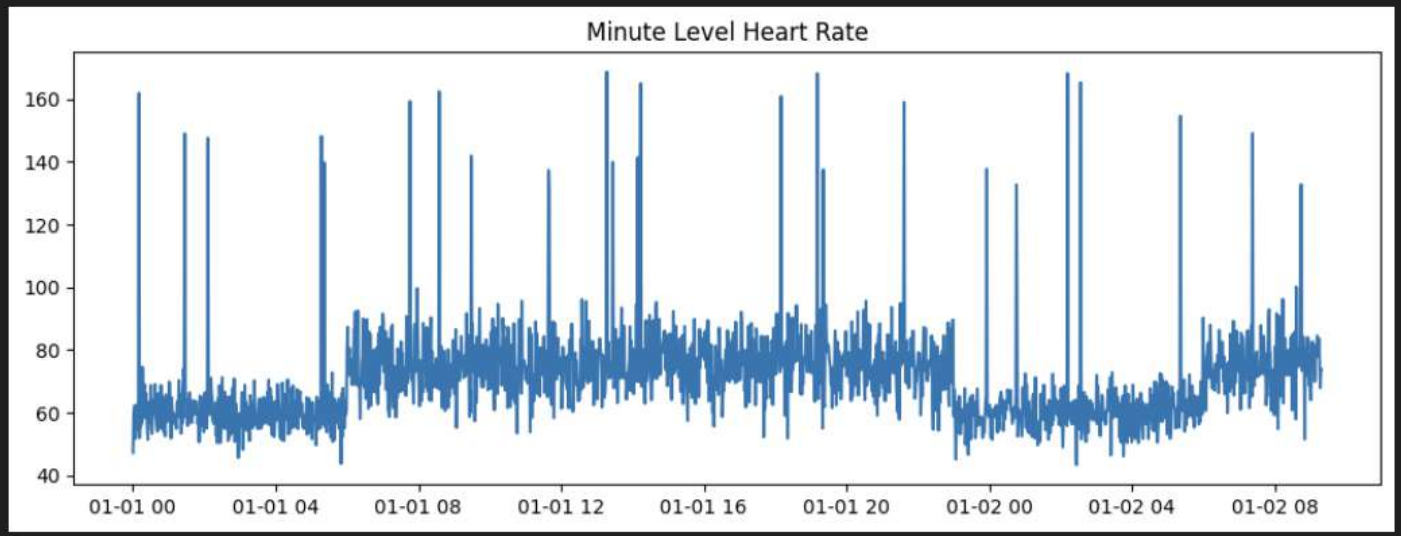
```python
df.isnull().sum()
```

[12] Python

```
heart_rate     0
steps          0
sleep_hours    0
dtype: int64
```

```python
plt.figure(figsize=(12,4))
plt.plot(df.index, df['heart_rate'])
plt.title("Minute Level Heart Rate")
plt.show()
```

[13] Python




```python
df_hourly = df.resample('h').mean()
```

[9] Python

```python
print("Original rows:", len(df))
print("Hourly rows:", len(df_hourly))
```

[22] Python

```
Original rows: 2000
Hourly rows: 34
```

[23]

```python
df_hourly['hr_rolling_mean'] = df_hourly['heart_rate'].rolling(window=24).mean()
```

Python

[24]

```python
df_hourly['z_score'] = stats.zscore(df_hourly['heart_rate'])
df_hourly['anomaly'] = df_hourly['z_score'].abs() > 2
```

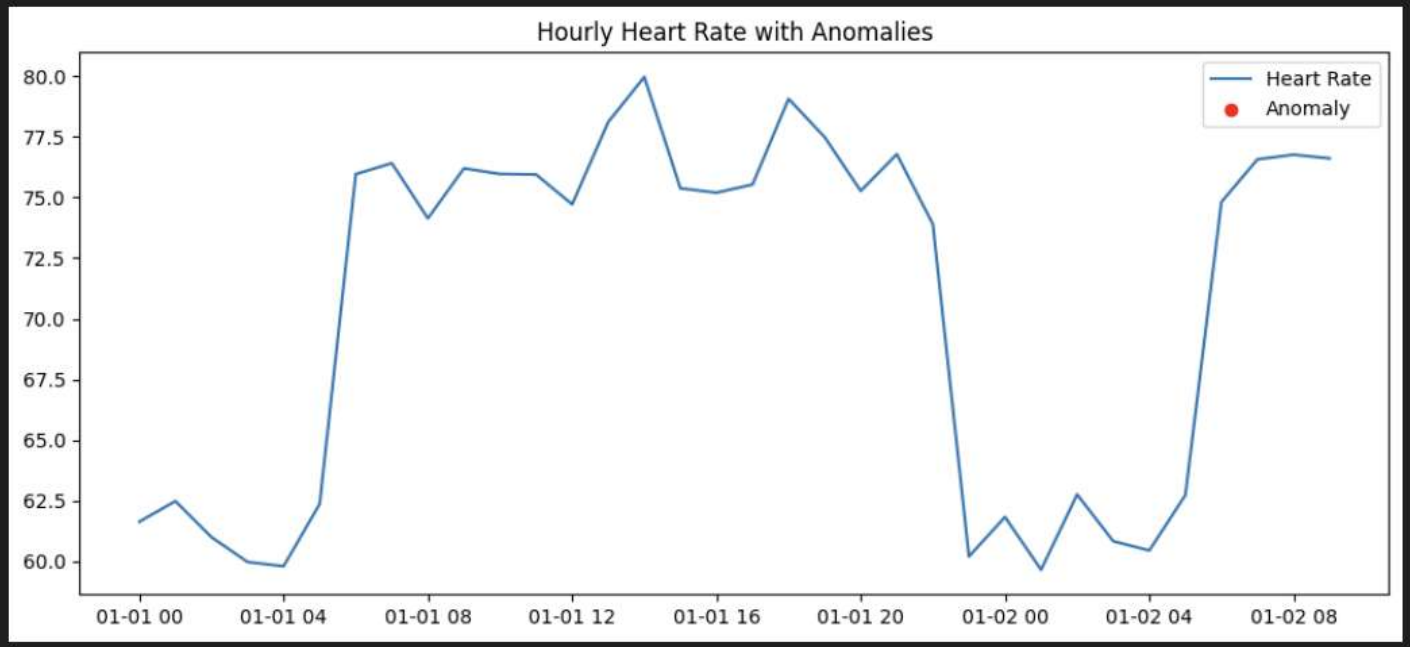
Python

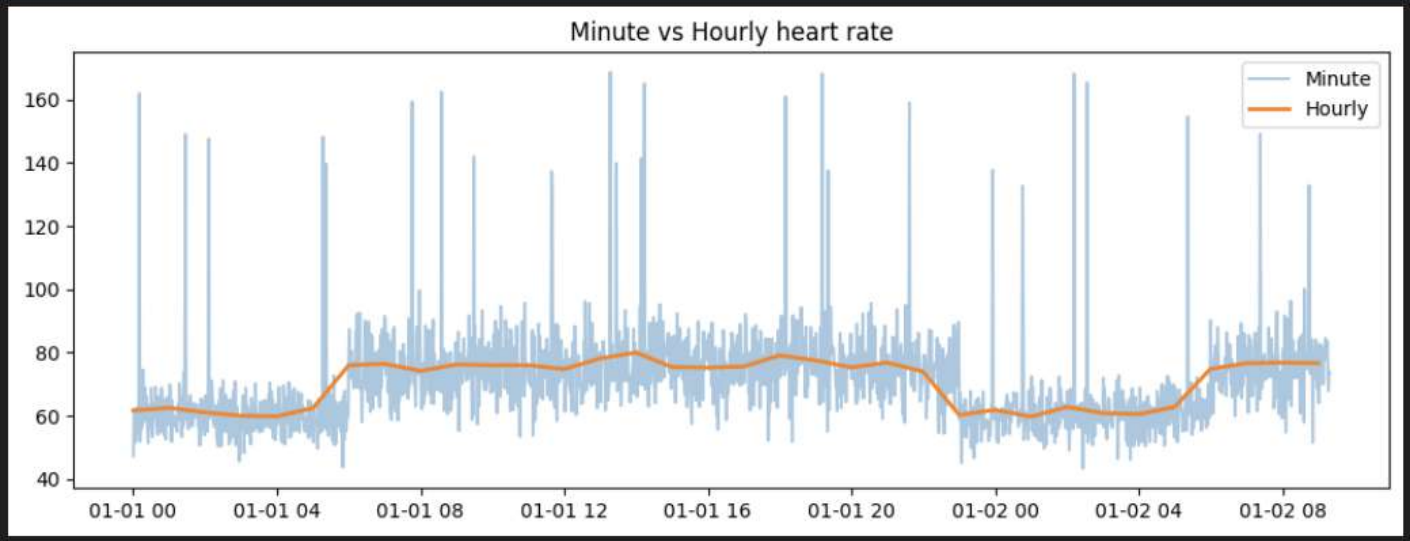
```python
plt.figure(figsize=(12,5))
plt.plot(df_hourly.index, df_hourly['heart_rate'], label='Heart Rate')

plt.scatter(
    df_hourly[df_hourly['anomaly']].index,
    df_hourly[df_hourly['anomaly']]['heart_rate'],
    color='red',
    label='Anomaly'
)

plt.legend()
plt.title("Hourly Heart Rate with Anomalies")
plt.show()
```

Python



```python
plt.figure(figsize=(12,4))
plt.plot(df['heart_rate'], alpha=0.4,label='Minute')
plt.plot(df_hourly['heart_rate'],linewidth=2,label='Hourly')
plt.legend()
plt.title("Minute vs Hourly heart rate")
plt.show()
```

[25] Python